

Flattening a Batch of Rank-Search-k Problems Project

Elias Smedegaard (plh150) and Jacob To (wtj903)

November 2025

Introduction (Elias)

This project concerns the `batch rank-search-k` algorithm on GPUs. Specifically, it deals with how to efficiently find the k th smallest element for a batch of arrays with varying sizes in Futhark. Since recursion and irregular arrays of arrays are not allowed in Futhark, we implement a flattened, non-recursive algorithm that uses a flattened input array instead of arrays of arrays. The goal for this is coalesced memory access and a competitive performance when compared to the baseline implementation in CUB which uses segmented sort. We also compared the performance to a sequential implementation using NumPy in Python.

In section 1, we start out by introducing the Futhark-like pseudocode of a rank-search algorithm using irregular arrays of arrays and recursion with expected $O(n)$ work. Afterwards, we introduce our flattened version derived using human reasoning in section 2. Section 3 is about the work-depth complexity of this version which finds that it has expected linear work and $O(\log(s_{max}))$ depth under the allowed assumption that the pivots remove half of the elements in each segment each iteration. Section 4 then compares the performance of this implementation to the baseline using CUB's segmented sort and NumPy sort in Python. Afterwards, section 5 introduces a generalized version of our flattened-by-human-reasoning implementation. Then, in section 6, we present how to flatten the three filters in the Futhark-like pseudocode in section 1 to use in a think-like-a-compiler flattening. At last, in section 7, we have a literature survey presenting the various selection algorithms.

How to run the code

Extract the project .zip file, copy the folder to the server, open a terminal on the server, and change into the project directory. Load the required modules and install the dependencies:

```
1 module load cuda
2 module load futhark
3 module load numpy
4 module load python/3.11.3
5 pip install futhark-data
```

Generate the Futhark datasets for benchmarking:

```
1 python gen_futhark_data.py
```

Build and run the CUB and Futhark tests and benchmarks:

```
1 make
```

Run the Python benchmark:

```
1 python sort.py
```

1. Presentation of the **rankSearch** pseudocode with expected $O(n)$ complexity (Jacob)

The high-level selection algorithm with linear expected work $O(n)$ is shown below as Futhark-like pseudocode **rankSearch**. This is also the algorithm presented in the assignment description. We index k from 1.

```
1 let rankSearch (k: i64) (A: []f32) : f32 =
2   let p          = random_element A
3   let A_lth_p    = filter (< p) A
4   let A_eqt_p    = filter (== p) A
5   let A_gth_p    = filter (> p) A
6
7   if k <= A_lth_p.length
8   then rankSearch k A_lth_p
9   else if k <= A_lth_p.length + A_eqt_p.length
10    then p
11    else rankSearch (k - A_lth_p.length - A_eqt_p.length) A_gth_p
12
13 let main [m] (ks: [m]i64) (As: [m][]f32) : [m]f32 =
14   map2 rankSearch ks As
```

The algorithm picks a random pivot p from A . It then partitions A into three separate arrays with elements less than p , equal to p , and greater than p .

The recursive calls functions like this:

1. If $k \leq |A_{<p}|$, it recurses on the array holding the elements less than the pivot ($< p$) with the same k .
2. If $k \leq |A_{<p}| + |A_{=p}|$, it returns p .
3. If k does not match the cases above, the answer lies in the array with elements greater than the pivot ($> p$), and it recurses there with k decreased by the sizes of the other two arrays.

At the bottom, **main** applies **rankSearch** to each pair (k_i, A_i) in the batch using **map2**.

This is not legal Futhark code. Futhark does not allow recursion and irregular arrays. Since **As** is an irregular array of arrays, and the algorithm includes recursion, a flattened, non-recursive implementation is required in practice. This is implemented in the following section using human reasoning.

2. Flattened batch rank-search-k using human reasoning in Futhark (Elias)

We implemented the batch version directly in Futhark in **rankSearch.fut** by following the human reasoning pseudocode we are suggested to focus on first. The function **rankSearchBatch** takes four arrays as input. **ks** holds the target k per segment, **shp** the current segment sizes, **III** maps each element in the flat value array **A** to its owning segment, and **A** contains the concatenation of all active segments. A single **for-loop** repeatedly updates these arrays until no elements are left to process, while **result** collects finished answers.

At the top of each iteration we compute segment ends with an exclusive scan of **shp** by first doing an inclusive **scan** on **shp** and then subtracting 1 from all elements. Then, we pick one

pivot per nonempty segment by selecting the last element in that segment and putting it in `pivots`.

```

1 let seg_ends = scan (+) 0 shp
2 let pivot_inds = map2 (\e s -> if s > 0 then e-1 else 0) seg_ends shp
3 let pivots = map2 (\i s -> if s > 0 then A[i] else 0f32) pivot_inds
  shp

```

For every element a we compare it with its segment pivot and form two flag arrays for $a < p$ and $a = p$. Two `hist` calls aggregate these flags by `II1` to produce per segment counts $|A_{<p}|$ and $|A_{=p}|$ in parallel.

```

1 let less_flags = map2 (\a j -> if a < pivots[j] then 1 else 0) A II1
2 let equal_flags = map2 (\a j -> if a == pivots[j] then 1 else 0) A II1
3 let owners64 = map i64.i32 II1
4 let less_counts = hist (+) 0 m owners64 less_flags
5 let equal_counts = hist (+) 0 m owners64 equal_flags

```

Using these counts, we decide a small integer kind per segment and store them all in `kinds`. -1 means the segment is already inactive since it has no elements, 0 means recurse on the less-than side, 1 means the pivot is the answer, and 2 means recurse on the greater-than side.

```

1 let kinds =
2   map (\(k, s, L, E) ->
3     if s == 0 then -1
4     else if k <= L then 0
5     else if k <= L+E then 1
6     else 2)
7   (zip4 ks shp less_counts equal_counts)

```

We then update `ks` and `shp` to match the chosen subproblem. If the k is in the left branch, k stays the same while the shape s of that segment is reduced to $|A_{<p}|$.

If the k is in the right branch, both the k and the shape s of that segment is reduced by $|A_{<p}| + |A_{=p}|$.

```

1 let ks' =
2   map (\(k, kind, L, E) ->
3     if kind == 0 then k
4     else if kind == 2 then k - L - E
5     else -1)
6   (zip4 ks kinds less_counts equal_counts)
7 let shp' =
8   map (\(kind, s, L, E) ->
9     if kind == 0 then L
10    else if kind == -1 then 0
11    else if kind == 1 then 0
12    else s - L - E)
13   (zip4 kinds shp less_counts equal_counts)

```

Segments that have hit the pivot case write the pivot into the `result` array.

```

1 let result' =
2   map (\i -> if kinds[i] == 1 then pivots[i] else result[i]) (iota m)

```

Then, we edit the current arrays that are not solved yet by keeping exactly the elements that matter for the next round. An element stays if its segment chose the left branch and $a < p$ or the right branch and $a > p$.

```

1 let keep_idx =
2   filter (\i ->
3     let j = II1[i]
4     let a = A[i]
5     let p = pivots[j]
6     let k = kinds[j]
7     let go_left  = k == 0 && a < p
8     let go_right = k == 2 && a > p
9     in go_left || go_right)
10  (indices A)

```

These `keep_idx` indices are applied to `A` and `II1`, which means it selects only the relevant elements in each subarray and preserves the original order inside each segment at the same time.

```

1 let A'   = map (\i -> A[i])   keep_idx
2 let II1' = map (\i -> II1[i]) keep_idx

```

The loop repeats as long as the length of `A` is above 0. When `A` is empty, the `result` array holds the answers for all segments.

This follows the intent of the flattening using human reasoning. There is no recursion, no irregular arrays of arrays, and only flat parallel operators such as `map`, `scan`, `hist`, and `filter`. Additionally, the `for-loop` plays the same role as recursion from the futhark-like pseudocode in section 1. Data and ownership are stored contiguously which gives coalesced access when counting and masking the subarrays. The implementation passes the provided tests we have created for it which is shown in a later section.

3. Discussion about work-depth complexity of our implementation (Elias)

We are allowed to assume that in expectation the pivot removes half of the elements in each iteration i . We let $n = n_0 = \sum \text{shp}$ be the initial length of the flattened array `A`. Then n_i is the remaining total elements at iteration i .

Each segment has some initial size s where $s_{\max}$ is the size of the longest initial segment (batch).

Work Complexity:

At each iteration i of the human-reasoned flattened `rankSearchBatch`, it runs 5 steps on the n_i left at that iteration.

Of the 5 steps, only creating the $< p$ and $== p$ flag arrays, computing the histograms, and applying the `keep_idx` filter involve a sweep over `A` or `II1`. All other steps involve smaller maps over the number of segments. Therefore, the work per iteration is $W(n_i) = O(n_i)$.

Under the assumption that the pivot removes half each time, then we have a geometric series where the first iteration processes at most n elements, then $n/2$, then $n/4$ and so forth. This geometric series converges to $2n$. As such, the total expected work is $O(n)$:

$$W(n) = \sum_i W(n_i) = O(n)$$

Consequently, in expectation, it is linear in the input size and, therefore, work efficient.

Instead, if the pivot always picks the current minimum or maximum in each segment, then worst case is $O(n^2)$.

Depth Complexity:

With the same assumption, every segment size s halves per pass. The loop stops when all segment sizes reach 0. Since each loop iteration plays the role of one recursion level, the number of iterations depends on the largest segment size s_{max} because the array of arrays is irregular. As such, we have the following depth D :

$$D = O(\log(s_{max}))$$

In the worst case, where the pivot picks the minimum and maximum in each segment, the depth is just the size of the longest segment $D = O(s_{max})$

4. Comparison to CUB and Python implementation (Jacob)

To compare our Futhark implementation with a naive sorting baseline, we use CUB’s segmented sort which is a batched sort across multiple, non-overlapping sequences of data items. The underlying sorting algorithm is undefined and depending on the segment size, it might be radix sort, merge sort or something else. We also include a Python implementation using NumPy as a sequential baseline, since each segment is sorted on the CPU without parallelism. The benchmark consists of a total input size of 128,000,000 elements. We vary the segmentation of this input by using different numbers of segments, which results in different segment sizes. This allows us to evaluate performance across both very large and very small per-segment workloads. The chosen segment sizes span from 1,000,000 elements per segment down to just 10 elements per segment.

The segment sizes is designed to align with warp-level execution behavior on GPU architectures, where many elements can be processed in parallel within a warp. By testing this full range, we can observe how each implementation scales as the size of each segment changes. For each configuration, we recorded two metrics, runtime in microseconds and memory throughput in GB/s. This allows us to compare not only absolute execution time but also the efficiency of memory utilization across different segment sizes.

The performance of our Futhark implementation using human-reasoning, the CUB’s baseline, and the sequential NumPy baseline are reported below in Table 1, Table 2, and Table 3, respectively and to look at the benchmark code, we would refer to `rankSearch.fut`, `cub_radix_sort.cu` and `sort.py`.

Items	Segments	Size of segment	Time [μ s]	Throughput [GB/s]
128,000,000	128	1,000,000	40,194.000	12.738
128,000,000	1,280	100,000	36,481.000	14.035
128,000,000	12,800	10,000	39,476.000	12.970
128,000,000	128,000	1,000	50,400.000	10.159
128,000,000	1,280,000	100	46,188.000	11.085
128,000,000	12,800,000	10	54,846.000	9.335

Table 1: `rankSearchBatch` performance for 128,000,000 items with segment sizes.

Items	Segments	Size of segment	Time [μ s]	Throughput [GB/s]
128,000,000	128	1,000,000	13,674.247	37.443
128,000,000	1,280	100,000	8,941.000	57.264
128,000,000	12,800	10,000	8,500.130	60.234
128,000,000	128,000	1,000	11,048.200	46.342
128,000,000	1,280,000	100	3,068.642	166.849
128,000,000	12,800,000	10	1,905.155	268.745

Table 2: CUB’s segmented sort performance for 128,000,000 items with segment sizes.

Items	Segments	Size of segment	Time [μ s]	Throughput [GB/s]
128,000,000	128	1,000,000	1,239,761.014	0.413
128,000,000	1,280	100,000	1,047,950.023	0.489
128,000,000	12,800	10,000	892,034.719	0.574
128,000,000	128,000	1,000	996,140.216	0.514
128,000,000	1,280,000	100	3,252,947.567	0.157
128,000,000	12,800,000	10	29,028,504.690	0.018

Table 3: NumPy sort performance for 128,000,000 items with segment sizes

The results show that CUB’s segmented sort achieves much higher throughput than both our human-reasoned Futhark implementation and the Python baseline. We also observe variability across segment sizes in terms of runtime. As noted above, segmented sort adapts its algorithm based on segment length, which likely explains the inconsistent results across different segment sizes in runtime.

The human-reasoned version performs significantly better than the purely sequential NumPy approach but still lags behind CUB, indicating that while our algorithm benefits from parallelism, it does not fully exploit hardware-level optimizations.

As expected, the NumPy implementation is slowest because it runs sequentially on the CPU and is limited by single-thread memory bandwidth rather than massive parallelism. Python and Futhark slow down when the number of segments becomes very large. For the Futhark implementation, this might be because tiny segments increase per-segment overhead and reduce per-block efficiency.

By contrast, CUB gets faster as the number of segments increases. A possible reason why this occurs could be that more segments let the GPU schedule many blocks and warps concurrently, which hides memory latency. Once a segment fits in a block tile, CUB’s segmented sort can keep most work in registers and shared memory, greatly reducing global reads and writes. If segments become extremely small, overhead can dominate and the gains falls off.

Validation and corner tests:

We validated correctness as follows. For the CUB and NumPy versions, we verified that each batch was sorted in ascending order. Because these rely on well-tested library sorting routines, we were confident in their correctness. For the Futhark implementation, we wrote and ran dedicated validation tests to confirm that the results matched the expected ordering. For example, we tested edge cases such as all inputs being empty which should return an empty `result` array. Additionally, we tested with the presence of a single empty segment in `A` and empty `III` with only the element `0i32` in `shp` and the element `1i32` in `ks` which should return the neutral element `0.0f32` in the `result` array. Furthermore, we also tested when segments had many duplicates. It passed all of our validation and corner tests.

5. Generalized implementation of batch rank-search-k in Futhark (Jacob)

The f32-based human-reasoned flattening of batch rank-search have been generalized to work with any element type `t` in the `rankSearch_gen.fut` file. Instead of hardcoding the comparators `<` and `==` for numbers, the algorithm now also takes two functions (predicates), `less: t -> t -> bool` and `equal: t -> t -> bool`, as well as a neutral value of the type `t`.

```
1 let rankSearchBatch [m][n] 't
2   (ks: [m]i32) (shp: [m]i32) (II1: [n]i32) (A: [n]t)
3   (less: t -> t -> bool)
4   (equal: t -> t -> bool)
5   (neutral: t) : [m]t
```

Every place where these numeric comparators were used to compare array elements `a` to the pivot `p`, these predicates are now called instead. For `>`, we have just used "`not less x y or equal x y`" such that one does not have to provide the greater-than predicate.

```
1 let greater (x: t) (y: t) : bool =
2   not (less x y || equal x y)
```

The neutral element is used in two places where we have to initiate with a value with the correct type, but never actually use it.

This is seen when a segment is empty and has size 0 but we still need to have a pivot element for it in the pivots array. Therefore, an empty segment is given the neutral element as pivot which is never used for comparisons.

```
1 let result = replicate m neutral
2 let pivots = map2 (\i s -> if s > 0 then A[i] else neutral) pivot_inds
   shp
```

The other place is when building the `result` array where every result is initialized with the neutral element.

```
1 let result = replicate m neutral
```

These changes means that using it is slightly different from the float-based version. For the original float case, one simply passes it the numeric comparators (`<`) and `==` and the neutral element 0 (`0f32`). For using it with a custom type, you first need to create your own predicates and neutral element. We have tested it with 2 different examples for this.

1. We have created a `months` type using the sum type in Futhark (`#Jan`, `#Feb`, ..., `#Dec`). A helper function maps each month to its calendar index. Additionally, we have made comparators where the less-than `lt_month` uses the calendar indices, the equal-to function is just (`==`), and the neutral element is `#Jan`.
2. We have done the same for card ranks (`#Ace`, `#Two`, ..., `#King`). The same idea is used here where we map to 1, 2, ..., 13, compare those indices in the less than function, use (`==`) for checking equality, and the neutral element is `#Ace`.

They all validate on our small tests.

6. Flattening the three filters in the think-like-a-compiler implementation (Elias)

In this section, we derive a compiler-like flattening of the three filters from the Futhark-like pseudocode in Section 1. We implement it as `partition3_seg` in `partition3.fut`. For context, we reference specific page numbers from the `L2-flattening.pdf` (Cosmin E. Oancea (2025)) slides since our approach mirrors the flattening of the `map` over nested Quicksort shown there. To place it in context, one first applies the flattening recipe on batched rank-search (p. 42). By doing this, the algorithm then resembles the Quicksort example (pp. 60-61): an outer `map` over an array of arrays whose body applies a per-segment partition. For batched rank-search, the three filters become a three-way partition (`partition3`). Flattening this step means flattening `map(partition3)` for the three predicates `<p`, `==p`, and `>p` where p is the pivot. The difficult step called out on p. 61 “flatten `map(partition2)`” therefore becomes “flatten `map(partition3)`” for us, which is exactly what `partition3_seg` implements.

Our implementation of `partition3_seg` follows the `partition2` implementation as closely as possible on p. 32 of the same slides: build predicate flags, run inclusive scans to create per-element positions, compute final indices, and perform a `scatter`.

As explained, we extend it to three partitions and to segmented arrays by using `mkFlagArray` to mark segment starts and `sgmScan` to compute per-segment positions, then producing `[<p|==p|>p]` inside every segment. The `mkFlagArray` and `sgmScan` implementations were given to us in earlier lab classes.

Our implementation first creates a `base` array holding the 0-based start index of each segment. This is done with a `scan` over the shape array `shp` to get `ends`, and then subtracting each element in `shp` from the corresponding element on the same index in `ends` to get `base`.

```
1 let ends  : [m]i32 = scan (+) 0 shp
2 let base  : [m]i32 = map2 (-) ends shp
```

Afterwards, we create a flag array with `mkFlagArray`.

```
1 let F0    : [n]bool = mkFlagArray (map i64.i32 shp) false (replicate m
    true)
2 let F     : [n]bool = F0 :> [n]bool
```

Then we use the predicates for each partition to create the predicate flag arrays which correspond to `cs` in the `partition2` implementation.

```
1 let lt_cs  : [n]bool = map2 (\a j -> a < piv[j]) A II1
2 let eq_cs  : [n]bool = map2 (\a j -> a == piv[j]) A II1
3 let gt_cs  : [n]bool = map2 (\lt eq -> not lt && not eq) lt_cs eq_cs
```

These are then type casted from `bool` to `i32`.

```
1 let t_lt   : [n]i32 = map i32.bool lt_cs
2 let t_eq   : [n]i32 = map i32.bool eq_cs
3 let t_gt   : [n]i32 = map i32.bool gt_cs
```

Now, we use the flag array `F` to do a segmented scan over the predicate flags which then counts up every time a new element satisfying the predicate is found.

```
1 let is_lt0 : [n]i32 = sgmScan (+) 0 F t_lt
2 let is_eq0 : [n]i32 = sgmScan (+) 0 F t_eq
3 let is_gt0 : [n]i32 = sgmScan (+) 0 F t_gt
```


This does not correspond to the `isT` and `isF` from `partition2` yet because we must add the per-segment partition start offsets. This is done by first computing the lengths of each less-than and equal-to partitions for each segment which is `L` and `E` respectively

```
1 let end_ix : [m]i32 = map2 (\b s -> if s > 0 then b + s - 1 else 0) base
  shp
2 let L : [m]i32 = map (\j -> if shp[j] > 0 then is_lt0[end_ix[j]] else 0)
  (iota m)
3 let E : [m]i32 = map (\j -> if shp[j] > 0 then is_eq0[end_ix[j]] else 0)
  (iota m)
```

Then, I define `start_lt = base`. The per-segment partition start offsets can now be computed by adding `start_lt` to `L` and `start_eq` to `E` element-wise with a `map2`.

```
1 let start_lt : [m]i32 = base
2 let start_eq : [m]i32 = map2 (+) start_lt L
3 let start_gt : [m]i32 = map2 (+) start_eq E
```

Finally, we can turn the per-element counts from the segmented scans `is_lt0`, `is_eq0`, and `is_gt0` into 1-based positions in the output array `B`. This is done by shifting them by the start index of the corresponding partition.

```
1 let is_lt : [n]i32 = map2 (\seg iL -> start_lt[seg] + iL) II1 is_lt0
2 let is_eq : [n]i32 = map2 (\seg iE -> start_eq[seg] + iE) II1 is_eq0
3 let is_gt : [n]i32 = map2 (\seg iG -> start_gt[seg] + iG) II1 is_gt0
```

Here, `is_lt`, `is_eq`, and `is_gt` are 1-based positions in `B`. Now, just like with `inds` in the `partition2` implementation, we can pick which of the three positions to use based on which predicate is satisfied and convert from 1-based to 0-based indexing by subtracting 1.

```
1 let inds : [n]i32 =
2 map (\(lt, eq, iLT, iEQ, iGT) ->
3   if lt then iLT - 1
4     else if eq then iEQ - 1
5       else iGT - 1)
6 (zip5 lt_cs eq_cs is_lt is_eq is_gt)
```

The final `scatter` then looks at array `A` and writes each element to its destination partition. This gives the order `[<p|==p|>p]` in every segment.

```
1 let B : [n]f32 = scatter (replicate n 0f32) (map i64.i32 inds) A
```

`B` is then returned together with the start indices of each partition `start_lt`, `start_eq`, and `start_gt`.

Using this partition `[<p|==p|>p]`, and the start index of each partition in each segment, one could use the `<p` or `>p` partition for each segment in the next iteration of the for-loop that replaces the recursion by also changing the `shp'` and `ks'` values like in the human-reasoning implementation.

7. Litterature survey of various rank-search-like algorithms (Jacob)

Median of medians algorithm:

The algorithm `rankSearch` introduced in Section 1 has worst-case time complexity $\Theta(n^2)$. This is due to poor selection of the pivot element, which leads to imbalanced partitions. A solution is to choose the pivot by dividing A into groups of five and taking the median of each group. Let the median of these medians be the new pivot. Then partition A into elements less than, greater than, or equal to the pivot, and recurse only on the side that can still contain the k -th smallest element. The running time can be divided into three parts:

1. Compute the $\frac{n}{5}$ medians, one from each group of five
2. Let B be the array of these medians and compute `median( $B$ )` to obtain the pivot
3. Partition A into elements less than, equal to, and greater than the pivot, then recurse with `Median of medians` only on the side that can still contain the k -th smallest element

Work Complexity:

It takes $\Theta(n)$ time to compute the $n/5$ medians. It takes $T(\lceil n/5 \rceil)$ time to recursively compute the median of medians. For (3), it takes $T(n')$ time to recursively call `Median of medians` on the side of the partition that contains k , where n' is the maximum number of elements on one side of the partition.

Estimation of the number of elements on one side of the partition:

If we define q as the median of medians, the number of medians that are greater than q is at least half of them. Here, there are two scenarios that lowers the amount of medians, the group of five elements that contains q and the possibly incomplete last group. So we can safely count at least:

$$|\text{number of medians} \geq q| = \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil - 2$$

The number of elements that are greater than or equal to q would be 3 per group of 5 elements minus the group containing q and possibly the last group. Hence:

$$|\text{Elements} \geq q| = 3 \left(\frac{1}{2} \left\lceil \frac{n}{5} \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$

Therefore the recursive subproblem has size at most:

$$n' = n - \frac{3n}{10} - 6 = \frac{7n}{10} + 6$$

The total running time satisfies:

$$T(n) = \Theta(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 6\right)$$

It can be shown that the solution is:

$$T(n) = O(n)$$

The derivations above are inspired by *Linear Time Selection* (2018).

Depth Complexity:

The term $T\left(\frac{7n}{10} + 6\right)$ dominates the recursion depth, since it shrinks the problem the slowest. For large n the additive constant only changes the depth by $O(1)$, so we approximate:

$$\frac{7n}{10} + 6 \approx \frac{7n}{10}$$

To determine the depth D of the recursion tree, we stop when the subproblem size reaches 1, since the median of a single-element array is equal to the element itself.

$$n \cdot \left(\frac{7}{10}\right)^D \leq 1$$

From here, we solve for D by taking the natural log of both sides:

$$\log n + D(\log 7 - \log 10) \leq \log(1)$$

$$D(\log 7 - \log 10) \leq \log(1) - \log n$$

$$D \leq \frac{\log(1) - \log n}{\log 7 - \log 10}$$

Here it can be observed that the depth complexity of **Median of medians** is:

$$D \sim O(\log n)$$

In a parallel setting this implies the span is $O(\log n)$ while the total work remains $O(n)$.

Introsselect algorithm:

A hybrid version of the algorithm introduced in section 1 and the algorithm above using the median of medians method is called **Introsselect**. It begins by using the selection approach from the naive algorithm. While this tends to work well in practice, there is a worst case scenario where the pivot choices remain poor and the recursion depth becomes large. At that point, **Introsselect** switches to the median of medians selection to ensure a guaranteed linear time bound.

In other words, the algorithm monitors its own recursion depth. If the depth exceeds a preset limit based on the size of the input, the pivot selection strategy changes from randomized to median of medians. This preserves good performance in typical cases while preventing the worst case behavior of pure randomized pivot selection. This section is inspired by David R. Musser (1997).

Floyd-Rivest algorithm:

The Floyd-Rivest algorithm is a divide and conquer algorithm, that shares many similarities with the naive algorithm introduced in section 1. It narrows the search to a small middle band before recursing, so the recursion typically hits a subproblem much smaller than n , giving expected linear time. It does this taking a sample of size $s \approx n^{2/3}$, Choose bounds u and v around k with gap $g \approx \sqrt{s \log n}$, make 3 partitions into elements $< u$, $[u, v]$ and elements $> v$. Recurse only on the needed part, usually the middle of expected size $O(n^{2/3} \log(n))$. The expected running time is linear and often faster than the algorithm in section 1 for large n . This section is inspired by Robert W. Floyd Ronald L. Rivest (1974).

Conclusion (Elias)

We presented a flattened, non-recursive batch rank-search-k algorithm for Futhark that replaces the irregular array of arrays and recursion with a single flat representation (`A`, `shp`, `III`, `ks`) and a for-loop to iteratively update the segments. With respect to flattening, it used a human-reasoned approach (count-by-owner and global keep-filter). The implementation achieves expected linear total work with depth $O(\log(s_{\max}))$ given the assumption that the pivot removes half of the elements in every segment for each iteration.

In practice, it outperforms a sequential NumPy baseline but does not match CUB’s segmented sort, which we use as a state-of-the-art baseline. The gap is most visible when there are many tiny segments.

Furthermore, we have shown that the algorithm can be generalized to work with arbitrary element types `t`. It then requires custom comparators that work on these types as well as a neutral element with that same type `t`.

Additionally, to showcase a different flattening method, we implemented how a flattening of `map(partition3)` would work in an implementation using think-like-a-compiler flattening.

In general, our implementations could be improved. Better pivot selection, such as a median-of-medians approach, can reduce variance and worst-case behavior. We could also narrow the search down using the approach from the Floyd–Rivest algorithm.

As such, our implementation does not live up to the state-of-the-art CUB baseline and does not have guaranteed $O(n)$ work complexity. The next step going forward would have been to implement the median-of-medians method of choosing the pivot.

References

- Cosmin E. Oancea (2025). *Parallel Basic Blocks and Flattening Nested Parallelism (L2-flattening.pdf)*. URL: <https://github.com/diku-dk/pmph-e2025-pub/blob/main/slides/L2-Flattenning.pdf> (visited on 11/07/2025).
- David R. Musser (1997). *Introspective Sorting and Selection Algorithms*. URL: <https://webpages.charlotte.edu/rbunescu/courses/ou/cs4040/introsort.pdf> (visited on 11/07/2025).
- Linear Time Selection* (2018). URL: <https://tildesites.bowdoin.edu/~ltoma/teaching/cs231/2018spring/Lectures/selection.pdf> (visited on 11/07/2025).
- Robert W. Floyd Ronald L. Rivest (1974). *Floyd–Rivest Algorithm*. URL: https://en.wikipedia.org/wiki/Floyd%E2%80%93Rivest_algorithm (visited on 11/07/2025).